Teoria grafurilor

Noțiuni introductive: terminologie și tipuri speciale de grafuri

Învață să descrii rețele de noduri și legături, să recunoști lanțuri, cicluri, arbori și grafuri cu proprietăți speciale

Curs introductiv



Ce vom parcurge

PLANUL PREZENTĂRII

Terminologie

- graf neorientat, graf orientat
- lanț, lanț elementar
- drum, drum elementar
- ciclu, ciclu elementar
- circuit, circuit elementar
- grad, graf parțial, subgraf
- conexitate, tare conexitate
- arbore, graf ponderat
- arbore parțial, arbore parțial de cost minim

Tipuri speciale de grafuri

- graf complet
- graf hamiltonian
- graf eulerian
- graf bipartit
- graf turneu

Idee de bază

Un graf modelează o mulțime de **obiecte** (noduri) și **legăturile** dintre ele (muchii sau arce): rețele sociale, hărți, rețele de calculatoare.

Graf neorientat

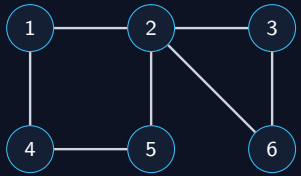
TERMINOLOGIE

Un **graf neorientat** este o pereche ordonată $G = (X, U)$, unde:

- X este o mulțime finită și nevidă, ale cărei elemente se numesc **noduri** (sau vârfuri);
- U este o mulțime de perechi neordonate de noduri distincte, numite **muchii**.

O muchie $[x, y]$ are **extremitățile** x și y. Nodurile x și y sunt **adiacente**, iar muchia este **incidentă** cu fiecare dintre ele.

Notații:
$n = |X|$ este numărul de noduri, iar $m = |U|$ este numărul de muchii. Muchia $[x, y]$ coincide cu $[y, x]$.



$X = \{1, 2, 3, 4, 5, 6\}$
$U = \{[1, 2], [2, 3], [1, 4], [2, 5], [3, 6], [4, 5], [2, 6]\}$
$n = 6, \quad m = 7$

Graf orientat

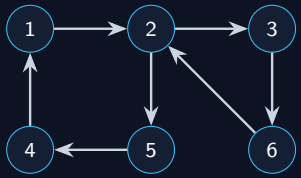
TERMINOLOGIE

Un **graf orientat** este o pereche ordonată $G = (X, U)$, unde:

- X este mulțimea finită și nevidă a **nodurilor**;
- U este o mulțime de perechi **ordonate** de noduri, numite **arce**.

Pentru arcul (x, y), nodul x este **extremitatea inițială**, iar y este **extremitatea finală**. Spunem că y este **succesor** al lui x, iar x este **predecesor** al lui y.

Reține:
Arcele (x, y) și (y, x) sunt diferite. Un graf orientat poate conține ambele arce între aceleași două noduri.



$X = \{1, 2, 3, 4, 5, 6\}$
$U = \{(1, 2), (2, 3), (3, 6), (6, 2), (2, 5), (5, 4), (4, 1)\}$

Lanț și lanț elementar

TERMINOLOGIE – graf neorientat

Un **lanț** este o succesiune de noduri $L = [x_1, x_2, \ldots, x_k]$ în care oricare două noduri consecutive sunt unite printr-o muchie: $[x_i, x_{i+1}] \in U$.

- **lungimea** lanțului este numărul de muchii, adică $k - 1$;
- nodurile x_1 și x_k sunt **extremitățile** lanțului;
- **lanț elementar**: toate nodurile lanțului sunt distincte.



Exemple (graful din figură)

Elementar: $4, 5, 2, 3, 6$ (lungime 4)

Neelementar: $1, 2, 3, 6, 2, 5$ (nodul 2 apare de două ori)

Drum și drum elementar

TERMINOLOGIE – graf orientat

Un **drum** este o succesiune de noduri $D = [x_1, x_2, \ldots, x_k]$ în care, pentru fiecare i, există arcul $(x_i, x_{i+1}) \in U$. Arcele se parcurg **în sensul lor**.

- **lungimea** drumului este numărul de arce, adică $k - 1$;
- **drum elementar**: toate nodurile drumului sunt distincte.



drum elementar



drum neelementar

Exemple (graful din figură)

Elementar: $1, 2, 3, 6$ (lungime 3)

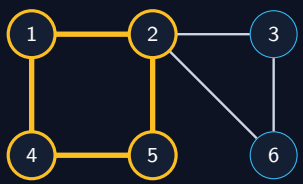
Neelementar: $1, 2, 3, 6, 2, 5$ (nodul 2 apare de două ori)

Ciclu și ciclu elementar

TERMINOLOGIE – graf neorientat

Un **ciclu** este un lanț $[x_1, \ldots, x_k]$ cu **muchii distincte**, în care primul și ultimul nod coincid: $x_1 = x_k$.

- **ciclu elementar**: singurele noduri care se repetă sunt cele două extremități ($x_1 = x_k$); toate celelalte sunt distincte;
- un ciclu elementar are cel puțin trei noduri distincte.



ciclu elementar: $1, 2, 5, 4, 1$

Exemple (graful din figură)

Elementar: $1, 2, 5, 4, 1$

Neelementar: $2, 3, 6, 2, 5, 4, 1, 2$ (nodul 2 apare de două ori, dar nicio muchie nu se repetă)

Circuit și circuit elementar

TERMINOLOGIE – graf orientat

Un **circuit** este un drum $[x_1, \ldots, x_k]$ cu **arce distincte**, în care $x_1 = x_k$.

- **circuit elementar**: singurele noduri care se repetă sunt cele două extremități; toate celelalte sunt distincte.

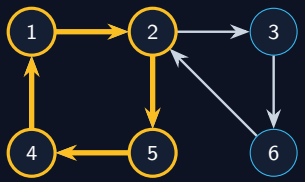
Exemple (graful din figură)

Elementar: $1, 2, 5, 4, 1$ și $2, 3, 6, 2$

Neelementar: $2, 3, 6, 2, 5, 4, 1, 2$

Reține:

Circuitul este pentru graful orientat ceea ce este ciclul pentru graful neorientat.



circuit elementar: $1, 2, 5, 4, 1$

Gradul unui vârf

TERMINOLOGIE

Definiție

Într-un graf neorientat se numește **grad** al unui vârf numărul de vârfuri adiacente cu acesta (sau numărul de muchii incidente cu acesta). Gradul unui vârf x se notează $d(x)$ (*degree*).

Observații:

- un vârf cu gradul 0 se numește **izolat**.
- un vârf cu gradul 1 se numește **terminal**.
- gradul maxim al unui vârf într-un graf cu n vârfuri este $n - 1$.

Teoremă:

- Suma gradelor tuturor vârfurilor este dublul numărului de muchii.

Consecințe:

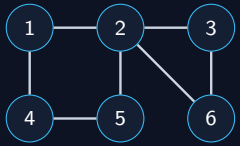
- Suma gradelor este un număr par.
- Numărul de vârfuri de grad impar este întotdeauna par.

Întrebare

Este posibil ca într-un grup de 5 persoane, fiecare persoană să aibă exact 3 prieteni?

Graf parțial și subgraf

TERMINOLOGIE



Graful inițial

$G = (X, U)$
$n = 6$ noduri, $m = 7$ muchii.



Graf parțial

$G' = (X, U')$, cu $U' \subseteq U$. Aceleași noduri; unele muchii sunt eliminate.



Subgraf

Se elimină unele noduri (aici nodul 4) împreună cu muchiile incidente cu ele.

Conexitate

TERMINOLOGIE – graf neorientat

Un graf neorientat este **conex** dacă pentru oricare două noduri x și y există un lanț între ele. Altfel, este **neconex**.

O **componentă conexă** este un subgraf conex maximal: nu mai poate fi extins cu alte noduri fără a-și pierde conexitatea.

Reține:
Un graf este conex exact atunci când are o singură componentă conexă. Pentru a le afla, se pornește un parcurgere (BFS sau DFS) din fiecare nod încă nevizitat.



graf conex



neconex: 3 componente

Tare conexitate

TERMINOLOGIE – graf orientat

Un graf orientat este **tare conex** dacă pentru oricare două noduri x și y există atât un drum de la x la y, cât și un drum de la y la x.

O **componentă tare conexă** este un subgraf tare conex maximal.

Reține:
Orice graf tare conex este conex (dacă ignorăm orientarea arcelor), dar reciproca nu este adevărată.



tare conex nu e tare conex

Al doilea graf nu are arcul $(4, 1)$: componentele tare conexe sunt $\{1\}$, $\{2, 3, 6\}$, $\{5\}$ și $\{4\}$.

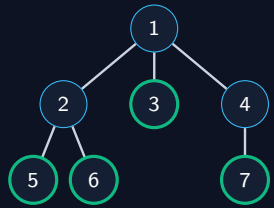
Arbore

TERMINOLOGIE – graf neorientat

Un **arbore** este un graf neorientat **conex** și **fără cicluri**.

Pentru un graf cu n noduri, următoarele afirmații sunt echivalente:

- graful este un arbore;
- graful este conex și are exact $n - 1$ muchii;
- între oricare două noduri există exact un lanț elementar;
- eliminând orice muchie, graful devine neconex.



$n = 7,\ m = 6$; frunzele (verde): $3, 5, 6, 7$

Frunze:

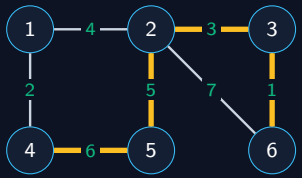
Nodurile de grad 1 se numesc **frunze**. Orice arbore cu cel puțin două noduri are cel puțin două frunze.

Graf ponderat

TERMINOLOGIE

Un **graf ponderat** (sau cu costuri) este un graf $G = (X, U)$ împreună cu o funcție $c : U \to \mathbb{R}$, care asociază fiecărei muchii (sau fiecărui arc) un număr numit **cost** (sau pondere).

Costul unui lanț (sau al unui drum) este suma costurilor muchiilor (arcelor) sale.



Exemplu

Lanțul $4, 5, 2, 3, 6$ are costul $6 + 5 + 3 + 1 = 15$.

Utilitate:

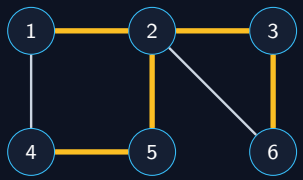
Costurile pot reprezenta distanțe, timp, preț sau capacitate, de exemplu într-o hartă rutieră.

Arbore parțial

TERMINOLOGIE

Un **arbore parțial** al unui graf neorientat $G = (X, U)$ este un graf parțial al lui G care este arbore: $T = (X, U')$, cu $U' \subseteq U$.

- are aceleași noduri ca G și exact $n - 1$ muchii;
- un graf are arbore parțial dacă și numai dacă este conex;
- un graf conex poate avea mai mulți arbori parțiali.



muchiile galbene formează un arbore parțial

Cum se obține?

Se elimină, pe rând, câte o muchie din fiecare ciclu, până când nu mai rămâne niciun ciclu. Aici: $m - (n - 1) = 7 - 5 = 2$ muchii eliminate (de exemplu $[1, 4]$ și $[2, 6]$).

Arbore parțial de cost minim

TERMINOLOGIE

Fie G un graf ponderat conex. Un **arbore parțial de cost minim** (APCM) este un arbore parțial al lui G pentru care **suma costurilor muchiilor** este cea mai mică dintre toți arborii parțiali ai grafului.



APCM de cost 15

Exemplu (graful din figură)

Muchiile $[3,6], [1,4], [2,3], [1,2], [2,5]$ au costul $1+2+3+4+5=15$.

Un alt arbore parțial, cu muchiile $[1,2], [2,3], [3,6], [2,5], [4,5]$, costă $4+3+1+5+6=19$.

Algoritmi clasici:

Kruskal adaugă muchiile în ordinea crescătoare a costurilor, dacă nu formează ciclu. **Prim** extinde un arbore pornind dintr-un nod, alegând mereu muchia de cost minim spre exterior.

PARTEA A II-A

Tipuri speciale de grafuri

graf complet · graf hamiltonian · graf eulerian · graf bipartit · graf turneu

Graf complet

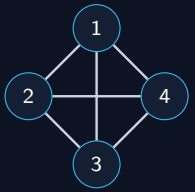
TIPURI SPECIALE

Un graf neorientat este **complet** dacă oricare două noduri distincte sunt adiacente. Graful complet cu n noduri se notează K_n.

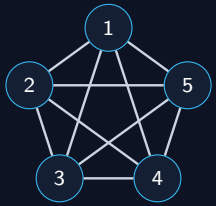
Proprietăți

- numărul de muchii: $m = \dfrac{n(n-1)}{2}$;
- fiecare nod are gradul $n-1$;
- K_n este conex pentru orice n.

Reține:
Graful complet este cel mai dens graf posibil: nu se mai poate adăuga nicio muchie.



K_4: 6 muchii



K_5: 10 muchii

Graf hamiltonian

TIPURI SPECIALE

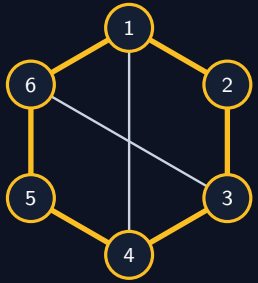
- **Lanț hamiltonian**: lanț elementar care trece prin toate nodurile grafului.
- **Ciclu hamiltonian**: ciclu elementar care trece prin toate nodurile grafului.
- **Graf hamiltonian**: graf care conține un ciclu hamiltonian.

Condiție suficientă (Dirac)

Dacă $n \geq 3$ și $d(x) \geq \frac{n}{2}$ pentru orice nod x, atunci graful este hamiltonian.

Reține:

Nu se cunoaște o condiție simplă care să fie și necesară, și suficientă. Problema este NP-completă.

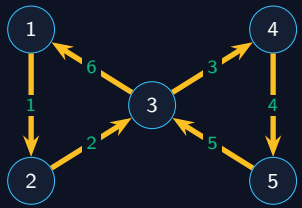


ciclu hamiltonian: $1, 2, 3, 4, 5, 6, 1$

Graf eulerian

TIPURI SPECIALE

- **Lanț eulerian**: lanț care conține fiecare muchie a grafului exact o dată.
- **Ciclu eulerian**: ciclu care conține fiecare muchie a grafului exact o dată.
- **Graf eulerian**: graf care conține un ciclu eulerian.



ciclu eulerian: $1, 2, 3, 4, 5, 3, 1$
(numerele verzi dau ordinea muchiilor;
$d(3) = 4$, restul au gradul 2)

Teoremă

Un graf $G = (X, U)$, fără vârfuri izolate, este eulerian dacă și numai dacă este conex și gradele tuturor vârfurilor sale sunt numere pare.

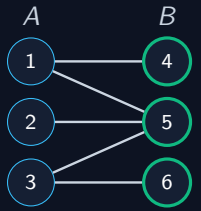
Graf orientat:

este eulerian dacă este conex și $d^-(x) = d^+(x)$ pentru orice nod x.

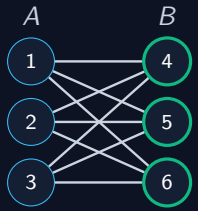
Graf bipartit

TIPURI SPECIALE

Un graf neorientat $G = (X, U)$ este **bipartit** dacă mulțimea nodurilor poate fi împărțită în două submulțimi disjuncte A și B ($X = A \cup B$, $A \cap B = \emptyset$) astfel încât **fiecare muchie** are o extremitate în A și cealaltă în B.



graf bipartit



$K_{3,3}$: 9 muchii

Caracterizări echivalente

- graful nu conține cicluri de lungime impară;
- nodurile pot fi colorate cu două culori, astfel încât nodurile adiacente să aibă culori diferite.

Bipartit complet $K_{p,q}$:
$|A| = p$, $|B| = q$, iar fiecare nod din A este adiacent cu fiecare nod din B. Are $p \cdot q$ muchii.

Graf turneu

TIPURI SPECIALE

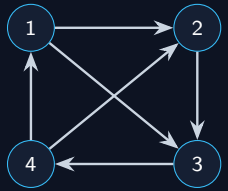
Un **graf turneu** este un graf orientat în care, între oricare două noduri distincte x și y, există **exact un arc**: fie (x, y), fie (y, x). Practic, se obține orientând fiecare muchie a unui graf complet.

Proprietăți

- numărul de arce: $m = \dfrac{n(n-1)}{2}$;
- orice turneu are un drum hamiltonian;
- un turneu tare conex are un circuit hamiltonian.

Exemplu intuitiv:

Un campionat în care fiecare echipă joacă o singură dată cu fiecare alta; arcul (x, y) înseamnă „x învinge y".



turneu cu 4 noduri și 6 arce;
circuit hamiltonian: $1, 2, 3, 4, 1$

Recapitulare

PE SCURT

Terminologie

- **neorientat** $\rightarrow$ muchii, lanț, ciclu;
- **orientat** $\rightarrow$ arce, drum, circuit;
- **elementar** $\rightarrow$ noduri distincte;
- **grad**: $\sum d(x) = 2m$;
- **conex** / **tare conex**: există lanț / drum în ambele sensuri;
- **arbore**: conex, fără cicluri, $m = n - 1$;
- **APCM**: arbore parțial cu suma costurilor minimă.

Tipuri speciale

- **complet** K_n: $\frac{n(n-1)}{2}$ muchii;
- **hamiltonian**: ciclu prin toate nodurile;
- **eulerian**: ciclu prin toate muchiile; grade pare;
- **bipartit**: noduri în două mulțimi, muchii doar între ele; fără cicluri impare;
- **turneu**: exact un arc între oricare două noduri.

```
Algorithm name: BFS/DFS
Input:
       g: a graph
       s: a starting vertex in g
Output:
       prints the vertices in a BFS/DFS traversal of g starting from s
Algorithm:
       Queue/Stack d  #Queue used for BFS, stack used for DFS
       Set visited
       d.push(s)
       while d is not empty:
              current = d.pop()
              print(current)
              for neighbor in g.get_neighbors(current):
                     if neighbor not in visited:
                            d.push(neighbor)
                     end_if
              end_for
       end_while
```

Figura: Pseudocodul general pentru BFS/DFS